

Министерство образования и науки Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ

“ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИТМО”

Факультет программной инженерии и компьютерной техники (ПИКТ)

Основы программной инженерии

Лабораторная работа № 4

Выполнил:

Серенко Егор Игоревич

Ровкова Анастасия Сергеевна

Р3216

Принял:

Коновалов Арсений Антонович

г. Санкт-Петербург

2026 г.

1. Задание

Лабораторная работа #4

Вариант 1984

Внимание! У разных вариантов разный текст задания!

1. Для своей программы из **лабораторной работы #3** по дисциплине "Веб-программирование" реализовать:

- MBeap, считающий общее число установленных пользователем точек, а также число точек, попадающих в область. В случае, если координаты установленной пользователем точки вышли за пределы отображаемой области координатной плоскости, разработанный MBeap должен отправлять оповещение об этом событии.
- MBeap, определяющий процентное отношение "промахов" к общему числу кликов пользователя по координатной плоскости.

2. С помощью утилиты JConsole провести мониторинг программы:

- Снять показания MBeap-классов, разработанных в ходе выполнения задания 1.
- Определить имя и версию ОС, под управлением которой работает JVM.

3. С помощью утилиты VisualVM провести мониторинг и профилирование программы:

- Снять график изменения показаний MBeap-классов, разработанных в ходе выполнения задания 1, с течением времени.
- Определить имя класса, объекты которого занимают наибольший объем памяти JVM; определить пользовательский класс, в экземплярах которого находятся эти объекты.

4. С помощью утилиты VisualVM и профилировщика IDE NetBeans, Eclipse или Idea локализовать и устранить проблемы с производительностью в **программе**. По результатам локализации и устранения проблемы необходимо составить отчет, в котором должна содержаться следующая информация:

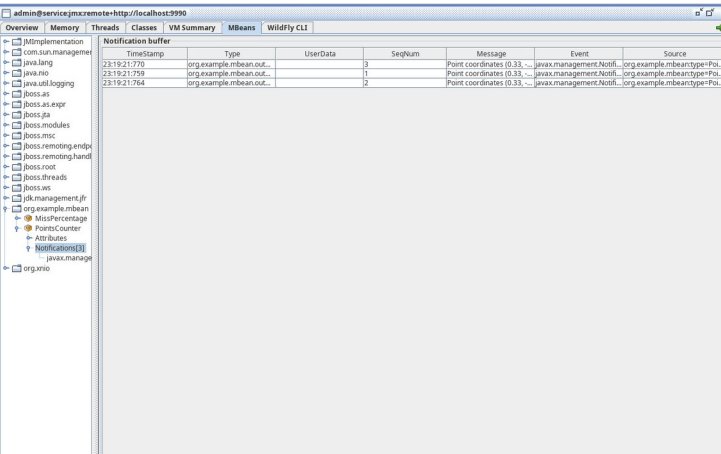
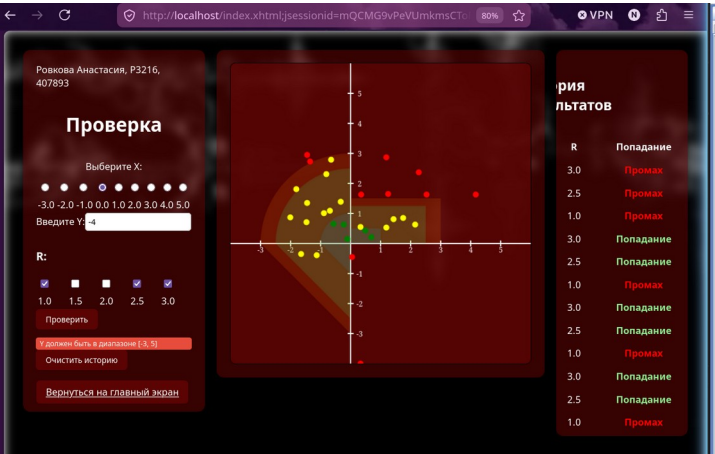
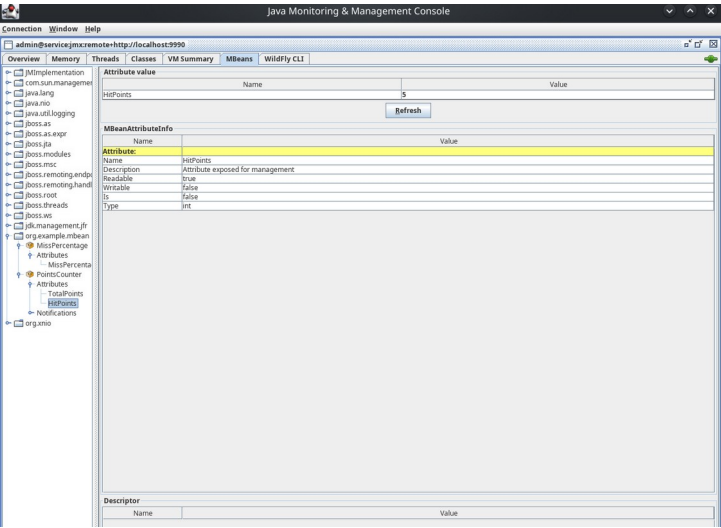
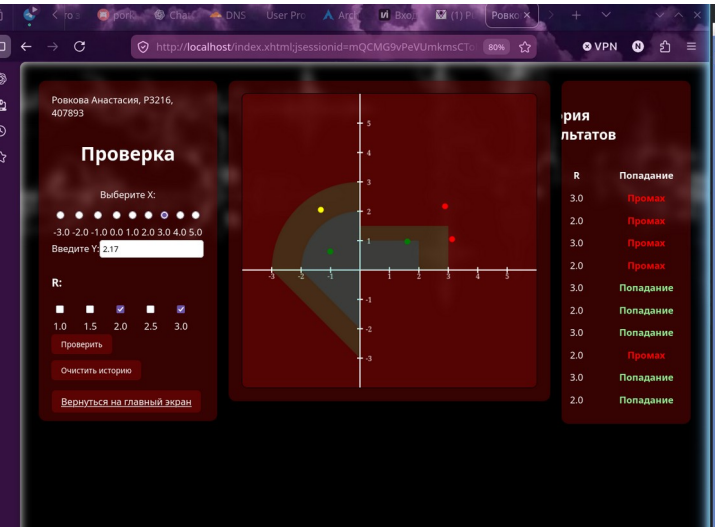
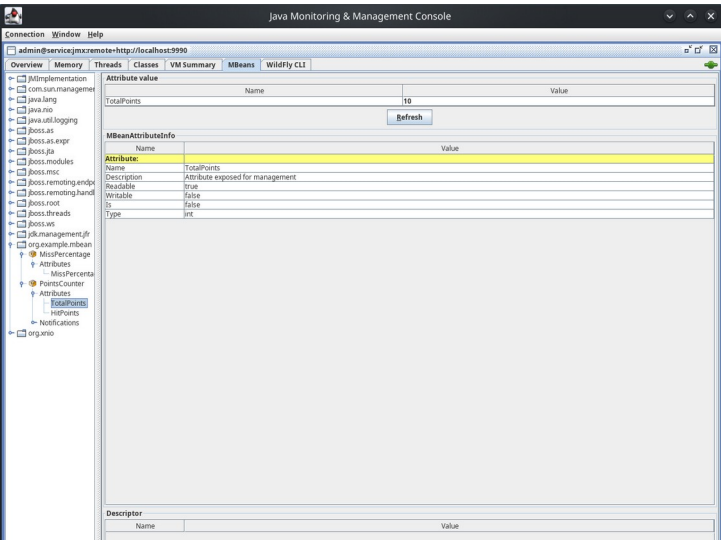
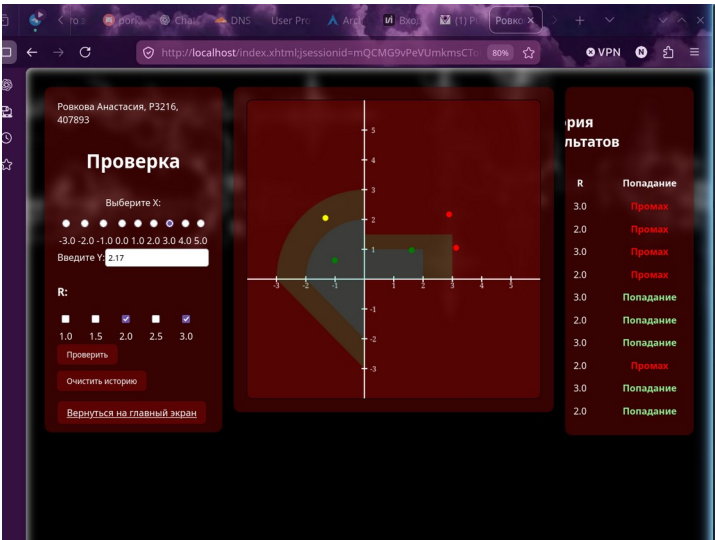
- Описание выявленной проблемы.
- Описание путей устранения выявленной проблемы.
- Подробное (со скриншотами) описание алгоритма действий, который позволил выявить и локализовать проблему.

Студент должен обеспечить возможность воспроизведения процесса поиска и локализации проблемы по требованию преподавателя.

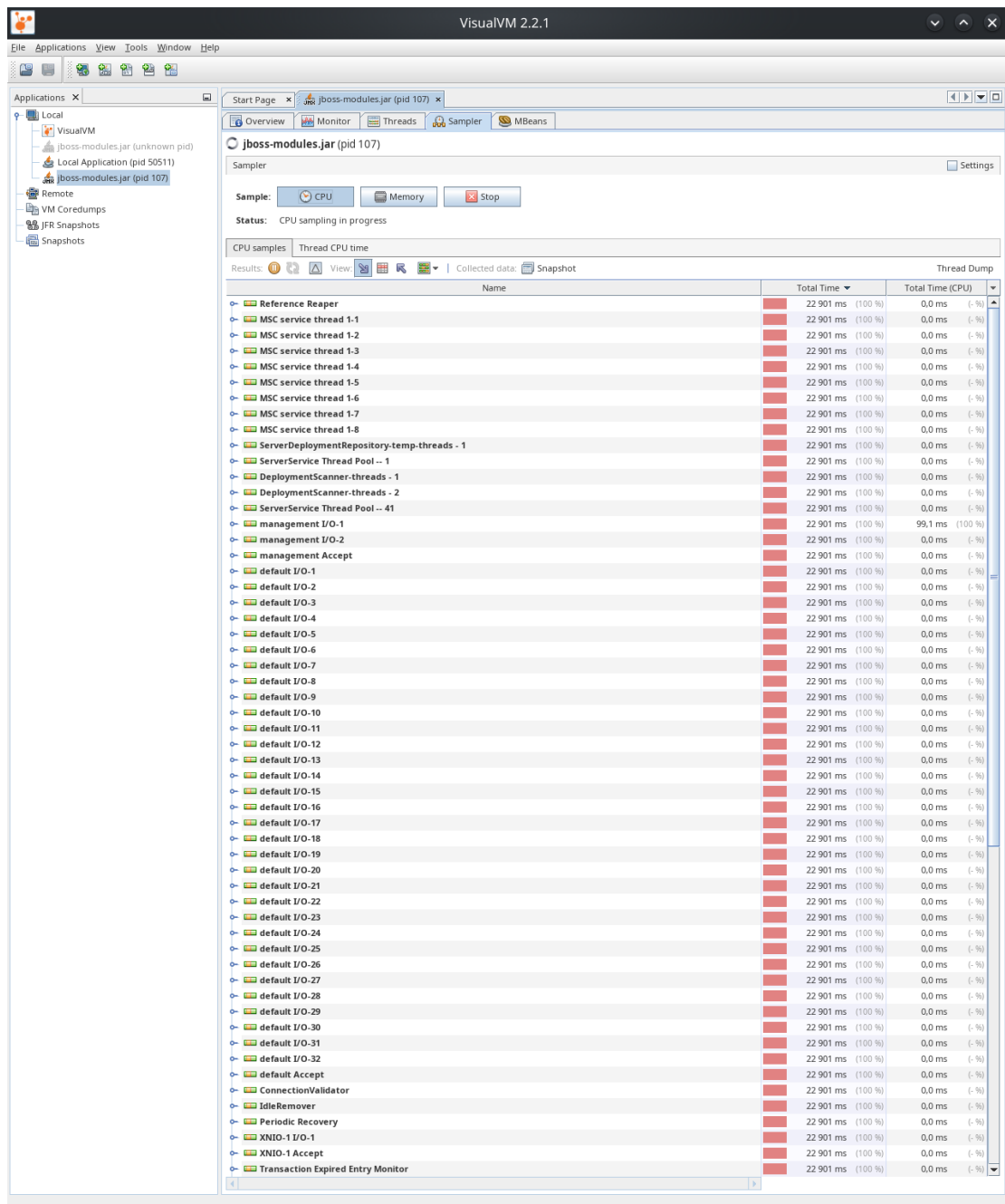
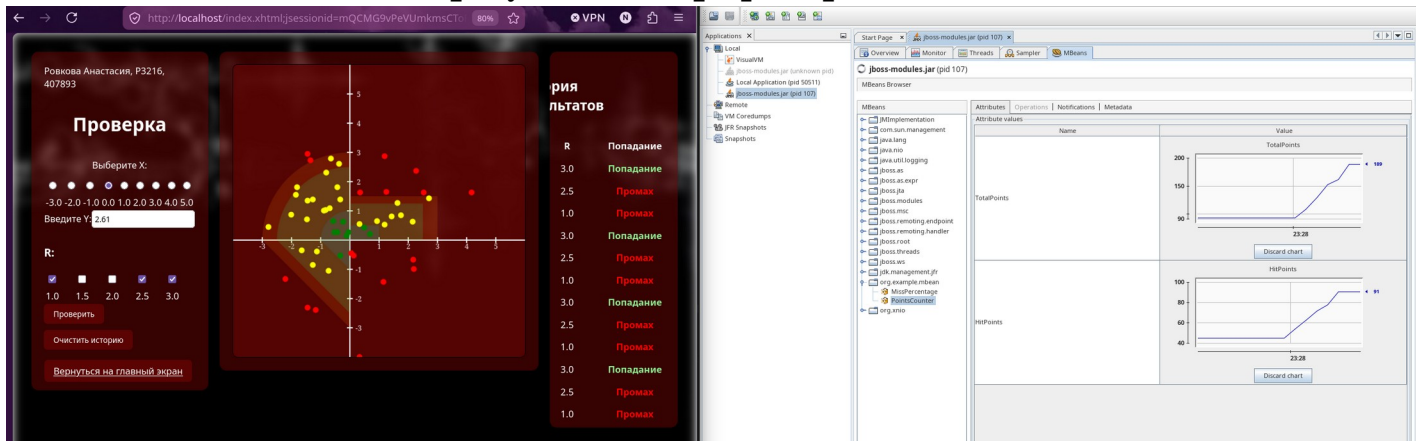
2. Исходный код разработанных MBeap-классов и сопутствующих классов.

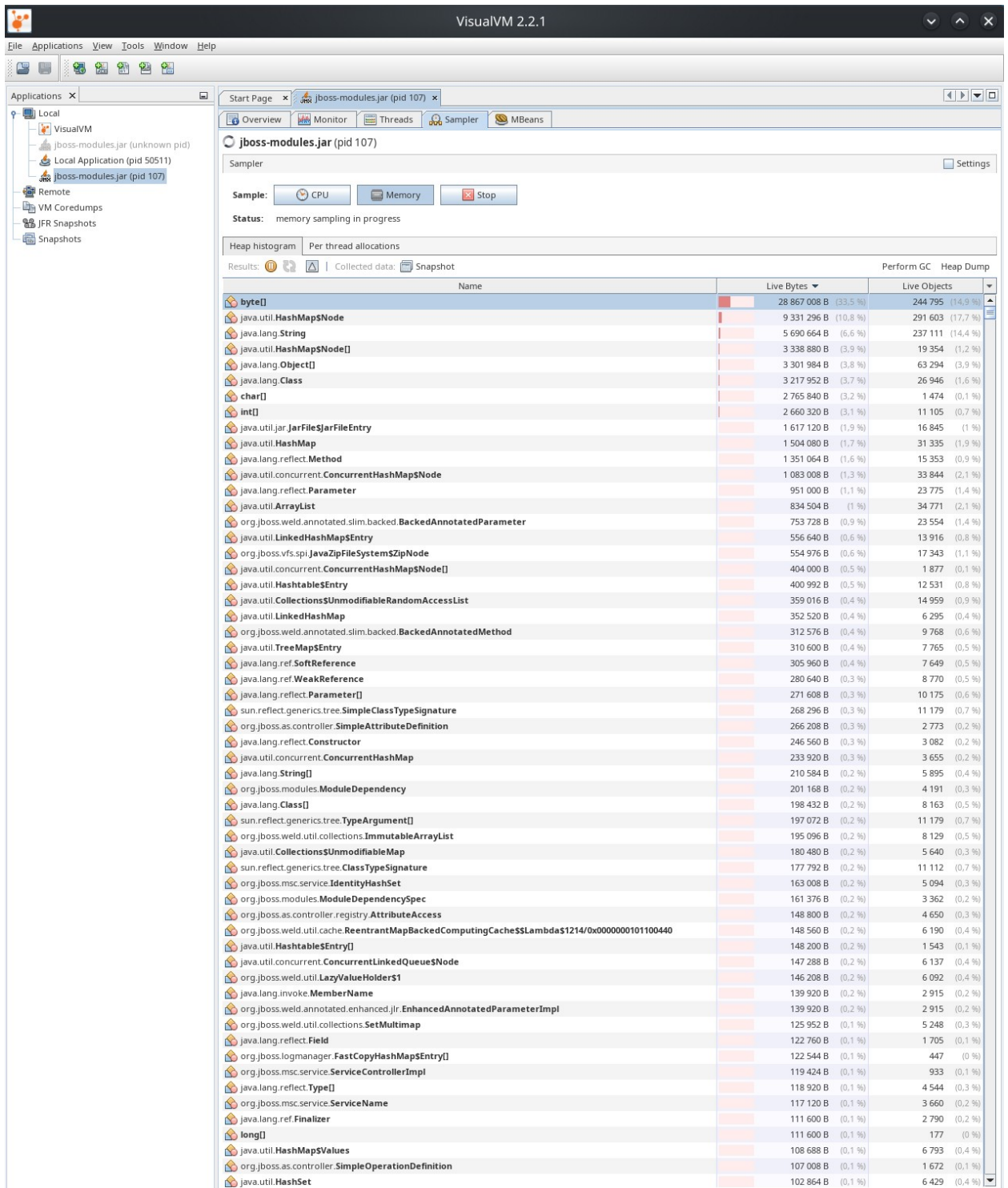
[https://github.com/Naswek/ITMO_labs/tree/main/
%D0%92%D0%B5%D0%B1/%D0%9B%D0%B0%D0%B1%203%20Java%D0%B0%D0%B1%20CriteriaAPI](https://github.com/Naswek/ITMO_labs/tree/main/%D0%92%D0%B5%D0%B1/%D0%9B%D0%B0%D0%B1%203%20Java%D0%B0%D0%B1%20CriteriaAPI)

3. Скриншоты программы JConsole со снятыми показаниями, выводы по результатам мониторинга.



4. Скриншоты программы VisualVM со снятыми показаниями, выводы по результатам профилирования.



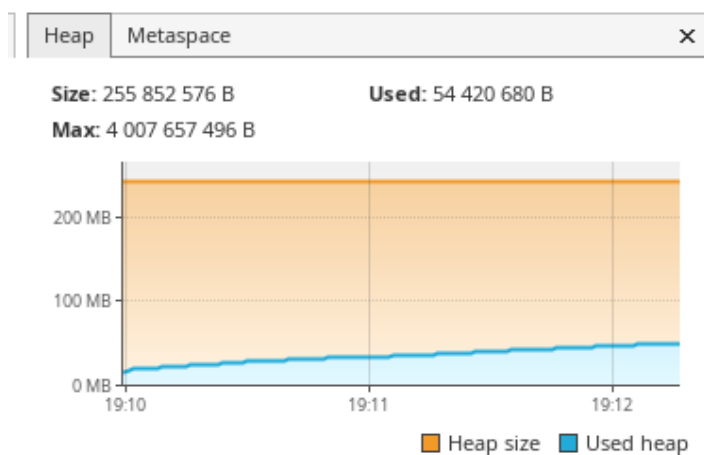
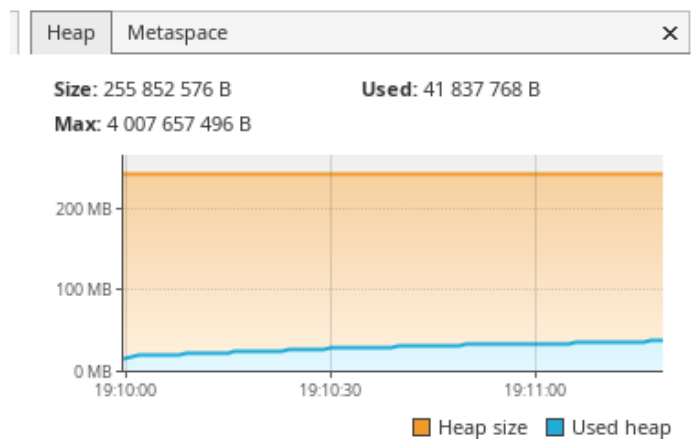
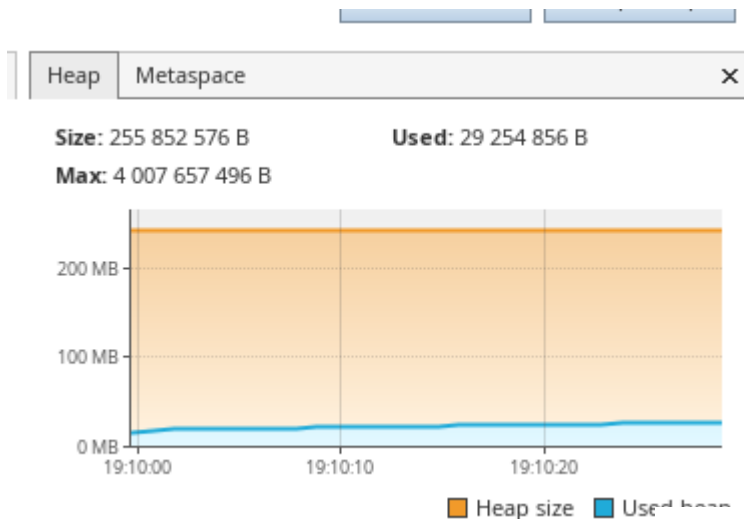


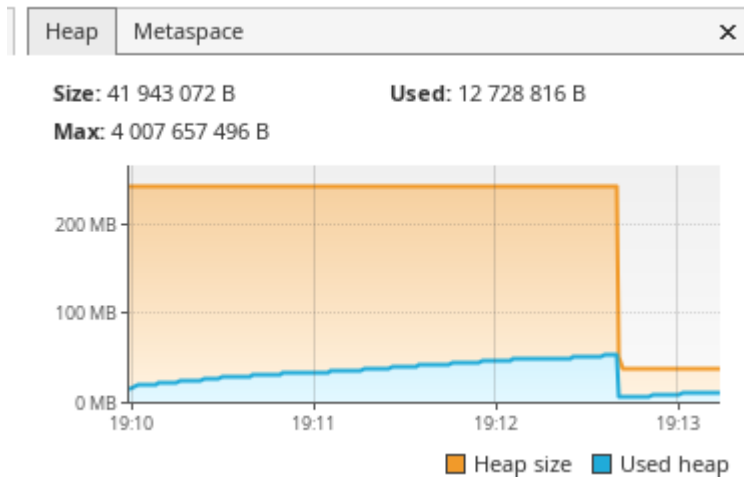
org.jboss.as.remoting.Namespace[]	48 B (0 %)	1 (0 %)
org.jboss.as.controller.access.management.WritableAuthorizerConfiguration\$MatchType	48 B (0 %)	2 (0 %)
org.example.beans.InputBean	48 B (0 %)	1 (0 %)
sun.security.rsa.RSAUtils\$KeyType	48 B (0 %)	2 (0 %)
org.wildfly.clustering.service.FunctionSupplierDependency	48 B (0 %)	2 (0 %)

org.postgresql.core.CachedQueryCreateAction	304 B (0 %)	19 (0 %)
org.jboss.weld.manager.BeanManagerImpl\$SLambda\$1211/0x00000001010d9840	304 B (0 %)	19 (0 %)
org.example.beans.ResultService	304 B (0 %)	19 (0 %)
org.jboss.security.plugins.JBossSecurityContextUtil	304 B (0 %)	19 (0 %)
org.postgresql.core.QueryExecutorCloseAction	304 B (0 %)	19 (0 %)

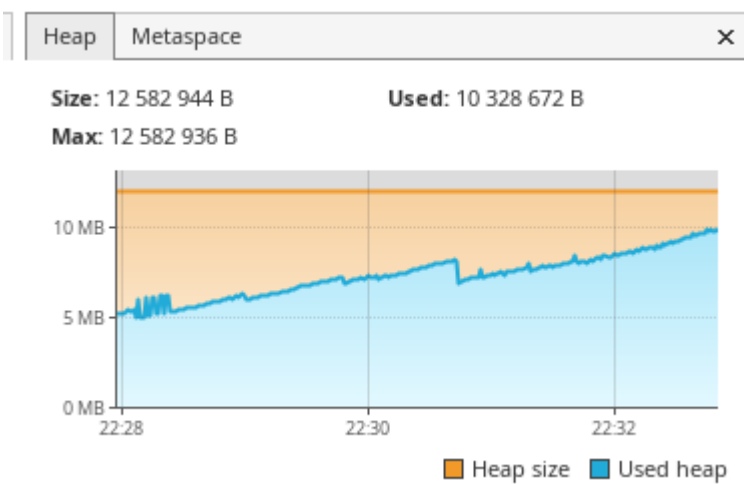
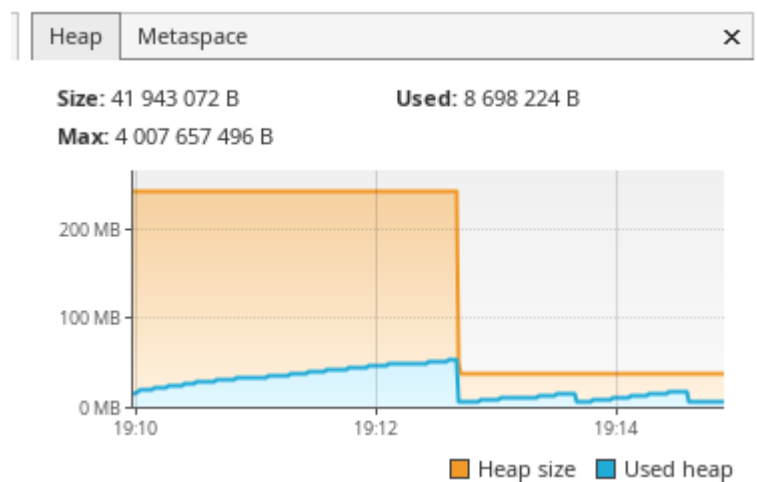
- Скриншоты программы VisualVM с комментариями по ходу поиска утечки памяти.

После запуска приложения и профайлера, можно заметить, как график использованной памяти непрерывно растет





Спустя п времени...



```
[java] Exception in thread "main" java.lang.OutOfMemoryError: Java heap space
[java]   at org.apache.xerces.util.NamespaceSupport.<init>(Unknown Source)
[java]   at org.cyberneko.html.HTMLScanner$ContentScanner.scan(Unknown Source)
[java]   at org.cyberneko.html.HTMLScanner.scanDocument(Unknown Source)
[java]   at org.cyberneko.html.HTMLConfiguration.parse(Unknown Source)
[java]   at org.cyberneko.html.HTMLConfiguration.parse(Unknown Source)
[java]   at org.apache.xerces.parsers.XMLParser.parse(Unknown Source)
[java]   at org.apache.xerces.parsers.DOMParser.parse(Unknown Source)
[java]   at com.meterware.httpunit.parsing.NekoHTMLParser.parse(NekoHTMLParser.java:41)
```


Пойдем смотреть какой класс в итоге скушал больше всего памяти

VisualVM 2.2.1

File Applications View Tools Window Help

Applications x

Local

- VisualVM
- jboss-modules.jar (unknown pid)
- Ant (pid 65894)
- Local Application (pid 37947)
- Main (pid 65941)
 - [heapdump] 00:59:38
 - [heapdump] 01:02:06
 - [heapdump] 01:02:57
- net.prominic.groovyjs.GroovyLanguageServer (p)

Remote

- VM CoreDumps
- JFR Snapshots
- Snapshots

Start Page x Main (pid 65941) x

[heapdump] 00:59:38 x [heapdump] 01:02:06 x [heapdump] 01:02:57 x

Overview Monitor Threads Sampler Profiler MBeans

Main (pid 65941)

Heap Dump

Summary

Heap

Size:	7 155 896 B
Classes:	3 389
Instances:	193 862
Classloaders:	112
GC Roots:	2 761
Objects Pending for Finalization:	0

Environment

System Linux (7.0.11-zen1-1-zen)

Architecture amd64 64bit

Java Home /usr/lib/jvm/java-17-openjdk

Java Version 17.0.19 2026-04-21

Java Name 3it Server VM (17.0.19+10, mixed mode, sharing)

Java Vendor Arch Linux

JVM Uptime 3 min 40 sec

Enabled Modules [show]

System Properties [show]

Classes by Number of Instances [view all]

java.util.TreeMap\$Entry	28 680 (14,8 %)
byte[]	25 253 (13 %)
java.lang.String	24 386 (12,6 %)
java.lang.Long	14 651 (7,6 %)
java.util.TreeMap	9 084 (4,7 %)

Classes by Size of Instances [view all]

byte[]	1 531 152 B (21,4 %)
java.util.TreeMap\$Entry	1 147 200 B (16 %)
java.lang.String	585 264 B (8,2 %)
java.util.TreeMap	436 032 B (6,1 %)
java.lang.Object[]	356 112 B (5 %)

Instances by Size [view all]

byte[]#467 : 98 473 items	98 496 B (1,4 %)
byte[]#1877 : 23 831 items	23 848 B (0,3 %)
byte[]#464 : 19 438 items	19 456 B (0,3 %)

Dominators by Retained Size [view all]

Retained sizes must be computed first.

Compute Retained Sizes

VisualVM 2.2.1

File Applications View Tools Window Help

Applications x

Local

- VisualVM
- jboss-modules.jar (unknown pid)
- Ant (pid 65894)
- Local Application (pid 37947)
- Main (pid 65941)
 - [heapdump] 00:59:38
 - [heapdump] 01:02:06
- net.prominic.groovyjs.GroovyLanguageServer (p)

Remote

- VM CoreDumps
- JFR Snapshots
- Snapshots

Start Page x Main (pid 65941) x

[heapdump] 00:59:38 x [heapdump] 01:02:06 x

Overview Monitor Threads Sampler

Main (pid 65941)

Heap Dump

Summary

Heap

Size:	6 276 272 B
Classes:	3 387
Instances:	166 831
Classloaders:	112
GC Roots:	2 773
Objects Pending for Finalization:	0

Environment

System Linux (7.0.11-zen1-1-zen)

Architecture amd64 64bit

Java Home /usr/lib/jvm/java-17-openjdk

Java Version 17.0.19 2026-04-21

Java Name 3it Server VM (17.0.19+10, mixed mode, sharing)

Java Vendor Arch Linux

JVM Uptime 2 min 49 sec

Enabled Modules [show]

System Properties [show]

Classes by Number of Instances [view all]

byte[]	24 732 (14,8 %)
java.lang.String	23 865 (14,3 %)
java.util.TreeMap\$Entry	20 548 (12,3 %)
java.lang.Long	10 586 (6,3 %)
java.lang.Object[]	7 004 (4,2 %)

Classes by Size of Instances [view all]

byte[]	1 490 976 B (23,8 %)
java.util.TreeMap\$Entry	821 920 B (13,1 %)
java.lang.String	572 760 B (9,1 %)
java.lang.Object[]	325 368 B (5,2 %)
java.util.TreeMap	312 000 B (5 %)

Instances by Size [view all]

byte[]#467 : 98 473 items	98 496 B (1,6 %)
byte[]#1877 : 23 831 items	23 848 B (0,4 %)
byte[]#464 : 19 438 items	19 456 B (0,3 %)

Dominators by Retained Size [view all]

Retained sizes must be computed first.

Compute Retained Sizes

Start PageMain (pid 83942)

OverviewMonitorThreadsSamplerProfilerMBeans[heapdump] 02:04:18[heapdump] 02:05:16[heapdump] 02:06:30[heapdump] 02:07:05[heapdump] 02:08:02[heapdump] 02:08:29

Main (pid 83942)

Heap DumpSummary

Heap

Size: 5 119 376 B

Classes: 3 387

Instances: 130 876

Classloaders: 111

GC Roots: 2 801

Objects Pending for Finalization: 0

Environment

SystemLinux (7.0.11-zen1-1-zen)

Architectureamd64 64bit

Java Home:/usr/lib/jvm/java-17-openjdk

Java Version:17.0.19 2026-04-21

Java Name:OpenJDK 64-Bit Server VM (17.0.19-10, mixed mode, sharing)

Java Vendor:Arch Linux

JVM Uptime:1 min 42 sec

Enabled Modules [show]

System Properties [show]

Classes by Number of Instances [view all]

byte[]23 896 (18.3 %)

java.lang.String23 021 (17.6 %)

java.util.TreeMap\$Entry9 848 (7.5 %)

java.lang.Object5 402 (4.1 %)

java.util.concurrent.ConcurrentHashMap\$Node5 239 (4 %)

Classes by Size of Instances [view all]

byte[]1 449 184 B (28.3 %)

java.lang.String552 504 B (10.8 %)

java.util.TreeMap\$Entry393 920 B (7.7 %)

java.lang.Object285 776 B (5.6 %)

java.util.concurrent.ConcurrentHashMap\$Node167 648 B (3.3 %)

Instances by Size [view all]

byte[]#48898 473 items98 496 B (1.9 %)

byte[]#187723 831 items23 848 B (0.5 %)

byte[]#47719 438 items19 456 B (0.4 %)

char[]#116 400 B (0.3 %)

char[]#4Count: 493[_response = com.meterware.servletunit.ServletUnitHttpResponse@274872f8]...16 400 B (0.3 %)

Dominators by Retained Size [view all]

Retained sizes must be computed first.

Compute Retained Sizes

Start PageMain (pid 83942)

OverviewMonitorThreadsSamplerProfilerMBeans[heapdump] 02:04:18[heapdump] 02:05:16[heapdump] 02:06:30[heapdump] 02:07:05[heapdump] 02:08:02[heapdump] 02:08:29

Main (pid 83942)

Heap DumpSummary

Heap

Size: 8 122 080 B

Classes: 3 389

Instances: 222 140

Classloaders: 113

GC Roots: 2 817

Objects Pending for Finalization: 0

Environment

SystemLinux (7.0.11-zen1-1-zen)

Architectureamd64 64bit

Java Home:/usr/lib/jvm/java-17-openjdk

Java Version:17.0.19 2026-04-21

Java Name:OpenJDK 64-Bit Server VM (17.0.19-10, mixed mode, sharing)

Java Vendor:Arch Linux

JVM Uptime:4 min 28 sec

Enabled Modules [show]

System Properties [show]

Classes by Number of Instances [view all]

java.util.TreeMap\$Entry37 133 (16.7 %)

byte[]25 798 (11.6 %)

java.lang.String24 926 (11.2 %)

java.lang.Long18 740 (8.4 %)

java.util.TreeMap11 770 (5.3 %)

Classes by Size of Instances [view all]

byte[]1 581 912 B (19.5 %)

java.util.TreeMap\$Entry1 485 320 B (18.3 %)

java.lang.String598 224 B (7.4 %)

java.util.TreeMap564 960 B (7 %)

java.lang.Long449 760 B (5.5 %)

Instances by Size [view all]

byte[]#48898 473 items98 496 B (1.2 %)

byte[]#187723 831 items23 848 B (0.3 %)

byte[]#47719 438 items19 456 B (0.2 %)

char[]#116 400 B (0.2 %)

char[]#4Count: 1296[_response = com.meterware.servletunit.ServletUnitHttpResponse@6ac43f7]...16 400 B (0.2 %)

Dominators by Retained Size [view all]

Retained sizes must be computed first.

Compute Retained Sizes

Start PageMain (pid 83942)

OverviewMonitorThreadsSamplerProfilerMBeans[heapdump] 02:04:18[heapdump] 02:05:16[heapdump] 02:06:30[heapdump] 02:07:05[heapdump] 02:08:02[heapdump] 02:08:29

Main (pid 83942)

Heap DumpSummary

Heap

Size: 8 777 608 B

Classes: 3 389

Instances: 242 953

Classloaders: 112

GC Roots: 2 797

Objects Pending for Finalization: 0

Environment

SystemLinux (7.0.11-zen1-1-zen)

Architectureamd64 64bit

Java Home:/usr/lib/jvm/java-17-openjdk

Java Version:17.0.19 2026-04-21

Java Name:OpenJDK 64-Bit Server VM (17.0.19-10, mixed mode, sharing)

Java Vendor:Arch Linux

JVM Uptime:4 min 56 sec

Enabled Modules [show]

System Properties [show]

Classes by Number of Instances [view all]

java.util.TreeMap\$Entry43 553 (17.9 %)

byte[]26 132 (10.8 %)

java.lang.String25 257 (10.4 %)

java.lang.Long21 765 (9 %)

java.util.TreeMap13 810 (5.7 %)

Classes by Size of Instances [view all]

java.util.TreeMap\$Entry1 742 120 B (19.8 %)

byte[]1 611 216 B (18.4 %)

java.util.TreeMap652 800 B (7.6 %)

java.lang.String606 168 B (6.9 %)

java.lang.Long522 120 B (5.9 %)

Instances by Size [view all]

byte[]#48898 473 items98 496 B (1.1 %)

byte[]#187723 831 items23 848 B (0.3 %)

byte[]#47719 438 items19 456 B (0.2 %)

char[]#116 400 B (0.2 %)

char[]#4Count: 1422[_response = com.meterware.servletunit.ServletUnitHttpResponse@1e545821]...16 400 B (0.2 %)

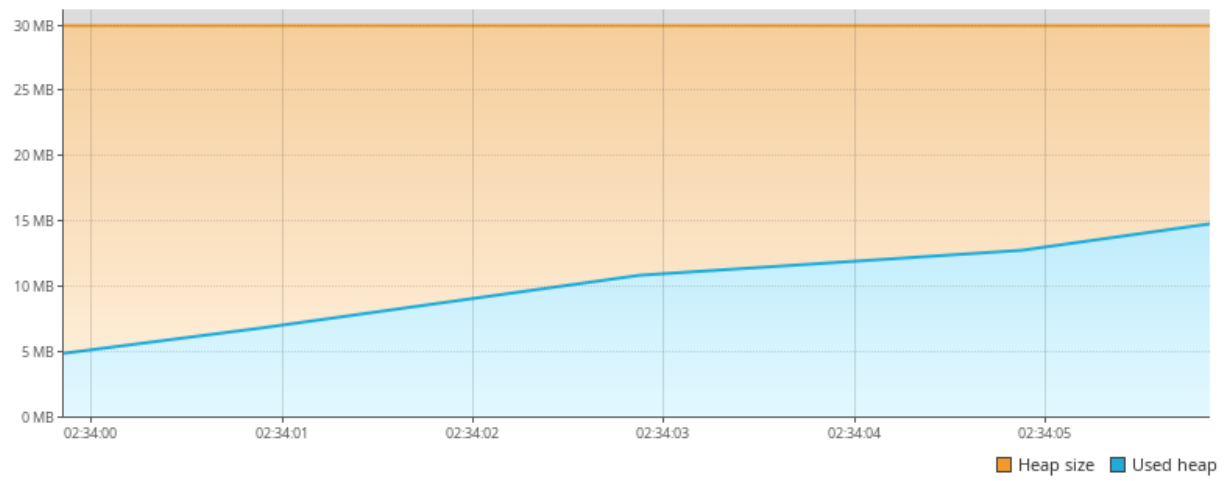
Dominators by Retained Size [view all]

Retained sizes must be computed first.

Compute Retained Sizes

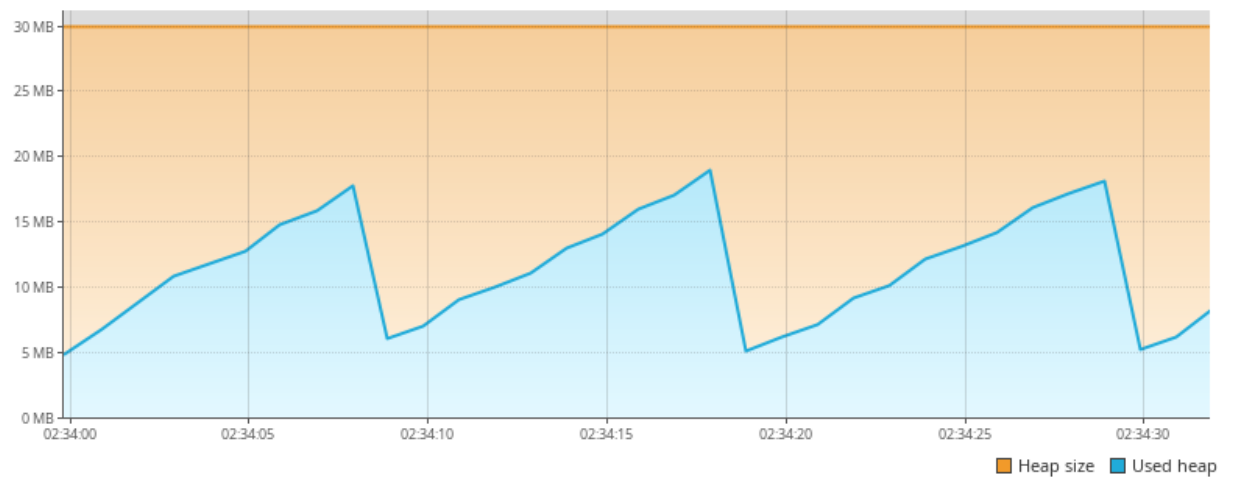
Size: 31 457 312 B
Max: 31 457 304 B

Used: 15 630 968 B



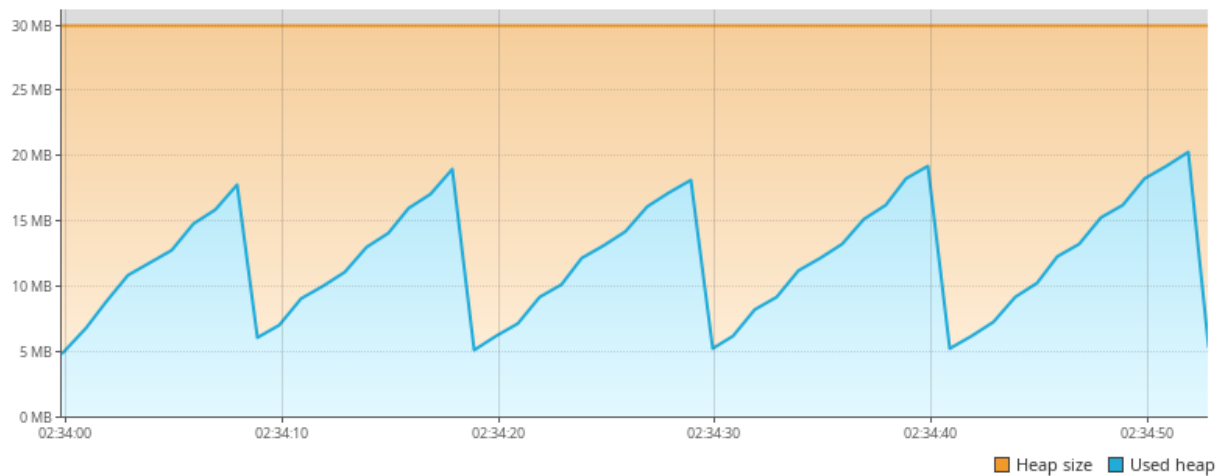
Size: 31 457 312 B
Max: 31 457 304 B

Used: 8 696 960 B



Size: 31 457 312 B
Max: 31 457 304 B

Used: 5 718 696 B



по графику видно, что идет утечка памяти, которая чистится GC, однако тем не менее, пики каждый раз растут больше и больше, зайдём в Sampler — Memory и посмотрим, что и сколько занимает:

Heap histogram		Per thread allocations	
Results:		Collected data: Snapshot	
Name		Live Bytes	Live Objects
byte[]		4 508 144 B (32.2 %)	58 602 (24.1 %)
char[]		1 494 208 B (10.7 %)	1 825 (0.8 %)
java.lang.String		964 296 B (6.9 %)	40 179 (16.5 %)
java.lang.reflect.Method		806 520 B (5.8 %)	9 165 (3.8 %)
java.lang.Object[]		784 048 B (5.6 %)	12 868 (5.3 %)
int[]		485 984 B (3.5 %)	3 247 (1.3 %)
java.lang.Class		454 408 B (3.2 %)	3 756 (1.5 %)
long[]		352 744 B (2.5 %)	1 326 (0.5 %)

-- ЭТО В ПИКЕ

Heap histogram		Per thread allocations		
Results:		Collected data: Snapshot		
Name		Live Bytes	Perform GC	Heap Dump
			Live Objects	
byte[]		2 577 568 B (28.9 %)	36 950	(21 %)
java.lang.String		742 032 B (8.3 %)	30 918	(17.6 %)
java.lang.Class		507 160 B (5.7 %)	4 227	(2.4 %)
char[]		481 568 B (5.4 %)	700	(0.4 %)
java.lang.Object[]		479 736 B (5.4 %)	8 052	(4.6 %)
int[]		386 424 B (4.3 %)	1 955	(1.1 %)
java.lang.reflect.Method		330 440 B (3.7 %)	3 755	(2.1 %)
java.util.concurrent.ConcurrentHashMap\$Node		206 016 B (2.3 %)	6 438	(3.7 %)
java.util.TreeMap\$Entry		196 680 B (2.2 %)	4 917	(2.8 %)

-- ЭТО В НИЗИНЕ

создадим head dump и посмотрим, что именон занимает так много места

Name		Count	Size	Retained
java.lang.Object[]		4 043 (4 %)	252 456 B (6.1 %)	1 248 504 B (30 %)
java.util.ArrayList		1 097 (1.1 %)	26 328 B (0.6 %)	503 456 B (12.1 %)
java.util.concurrent.ConcurrentHashMap		448 (0.4 %)	28 672 B (0.7 %)	473 952 B (11.4 %)
java.util.concurrent.ConcurrentHashMap\$Node[]		102 (0.1 %)	52 128 B (1.3 %)	454 296 B (10.9 %)
java.util.HashMap		549 (0.5 %)	26 352 B (0.6 %)	423 168 B (10.2 %)
java.util.Map\$Node[]		811 (0.8 %)	88 472 B (2.2 %)	416 720 B (10.3 %)

Залезем в Object[]

Main (pid 96493)			
Heap Dump			
Objects Preset: All Objects Aggregation: Details: Preview Fields References GC Root Hierarchy			
Name		Count	Retained
java.lang.Object[]		6 750 (4 %)	3 720 768 B (56.3 %)
java.lang.Object[]#8336 1 001 items			1 933 168 B (29.2 %)
<items>			
<references>			
queue in com.sun.jmx.remote.internal.ArrayQueue#1			1 933 200 B (29.2 %)
queue in com.sun.jmx.remote.internal.ArrayNotificationBuffer#1 [GC root - java frame]			1 933 384 B (29.2 %)
java.lang.Object[]#722 474 items			675 960 B (10.2 %)
<items>			
<references>			
elementData in java.util.ArrayList#1100 434 elements			675 984 B (10.2 %)
classes in jdk.internal.loader.ClassLoaders\$AppClassLoader#1 [GC root - java frame, JNI global]			743 840 B (11.3 %)
java.lang.Object[]#4261 4 164 items			615 856 B (9.3 %)
<items>			
<references>			
elementData in java.util.ArrayList#30 8 161 elements			615 880 B (9.3 %)
static_errorMessages in class com.meterware.httpunit.javascript.JavaScript : javascript			615 920 B (9.3 %)

-- первые объект jmx буффер

второй объект -

третий объект — уже коллекция, которая используется в программе
попробуем её найти

```

naswek@kinder-pingui ~/I/O/Л/HttpUnit (main)> grep -R "_errorMessages" .
grep: ./build/classes/com/meterware/httpunit/javascript/JavaScript.class: двоичный файл совпадает
./src/com/meterware/httpunit/javascript/JavaScript.java:    private static ArrayList _errorMessages = new ArrayList();
./src/com/meterware/httpunit/javascript/JavaScript.java:        _errorMessages.clear();
./src/com/meterware/httpunit/javascript/JavaScript.java:        return (String[]) _errorMessages.toArray( new String[ _errorMessages.size() ] );
./src/com/meterware/httpunit/javascript/JavaScript.java:        _errorMessages.add( errorMessage );
naswek@kinder-pingui ~/I/O/Л/HttpUnit (main)> grep -R "clearError" .
grep: ./build/classes/com/meterware/httpunit/HttpUnitOptions.class: двоичный файл совпадает
grep: ./build/classes/com/meterware/httpunit/HttpUnitOptions$1.class: двоичный файл совпадает
grep: ./build/classes/com/meterware/httpunit/javascript/JavaScriptEngineFactory.class: двоичный файл совпадает
grep: ./build/classes/com/meterware/httpunit/javascript/JavaScript.class: двоичный файл совпадает
grep: ./build/classes/com/meterware/httpunit/scripting/ScriptingEngineFactory.class: двоичный файл совпадает
./src/Main.java:        options.clearErrorMessages();
./src/com/meterware/httpunit/HttpUnitOptions.java:    getScriptingEngine().clearErrorMessages();
./src/com/meterware/httpunit/HttpUnitOptions.java:    public void clearErrorMessages() {}
./src/com/meterware/httpunit/javascript/JavaScriptEngineFactory.java:    public void clearErrorMessages() {
./src/com/meterware/httpunit/javascript/JavaScriptEngineFactory.java:        JavaScript.clearErrorMessages();
./src/com/meterware/httpunit/javascript/JavaScript.java:    static void clearErrorMessages() {
./src/com/meterware/httpunit/scripting/ScriptingEngineFactory.java:    public void clearErrorMessages();
naswek@kinder-pingui ~/I/O/Л/HttpUnit (main)>

```

```

ОПН/ИР4/HttpUnit/src/com/meterware/httpunit/javascript/JavaScript.java
Search...
36  * This class is the Rhino-compatible implementation of the JavaScript DOM objects.
37  *
38  * @author <a href="mailto:russgold@httpunit.org">Russell Gold</a>
39  **/
40  public class JavaScript {
41
42      private final static Object[] NO_ARGS = new Object[0];
43
44      private static boolean _throwExceptionsOnError = true;
45
46      private static ArrayList _errorMessages = new ArrayList();
47

```

static ArrayList не чистится, из-за чего возникает утечка

```

453  /**
454      * Clears the accumulated script error messages.
455      */
456  public static void clearScriptErrorMessages() {
457      getScriptingEngine().clearErrorMessages();
458  }
459

```

Пофиксим тем, что создам образец класса, который использует внутри метод clearErrorMessages, и будет вызывать его метод по очистке массива с ошибками

```

37  public static void main(String[] args) {
38      try {
39          HttpUnitOptions.setExceptionsThrownOnScriptError(false);
40          ServletRunner sr = new ServletRunner();
41          sr.registerServlet("myServlet", HelloWorld.class.getName());
42          ServletUnitClient sc = sr.newClient();
43          int number = 1;
44          WebRequest request = new GetMethodWebRequest("http://test.meterware.com/myServlet");
45          while (true) {
46              WebResponse response = sc.getResponse(request);
47              System.out.println("Count: " + number++ + response);
48              HttpUnitOptions.clearScriptErrorMessages();
49              java.lang.Thread.sleep(200);
50          }

```

запускаем ant compile...

посмотрим монитор:



теперь графики более ровные

3. Выводы по работе.

В ходе лабораторной работы были реализованы MBean-компоненты для мониторинга количества точек, попаданий в область и процента промахов. С помощью JConsole и VisualVM было выполнено подключение к JVM, просмотрены значения MBean, получена информация об ОС и проанализировано использование памяти. Также были изучены основы профилирования Java-приложения и поиска проблем производительности. Работа позволила закрепить навыки мониторинга приложения во время выполнения и анализа его состояния с помощью JMX-инструментов.